

SQL Query Optimization

Links:

<https://www.geeksforgeeks.org/sql/best-practices-for-sql-query-optimizations/>

<https://www.datacamp.com/blog/sql-query-optimization>

Some of the points on Query Optimization

1. -- Inefficient

```
SELECT * FROM Customers;
```

-- Efficient

```
SELECT CustomerID, FirstName, LastName FROM Customers;
```

2. Optimize JOIN Operations: Choose the most appropriate JOIN type for your needs. INNER JOIN is generally more efficient than LEFT JOIN when only matching data is required. Ensure **JOIN conditions are indexed**.

3. Refine WHERE Clauses and Filtering: Use WHERE clauses to filter data as early as possible in the query execution. This reduces the number of rows processed by subsequent operations. Avoid using SELECT DISTINCT if the same result can be achieved through better filtering.

4. Utilize Stored Procedures and Execution Plans: Stored procedures, being pre-compiled, can offer performance benefits for frequently executed queries. Analyze query execution plans to identify bottlenecks and inefficient operations, such as full table scans or suboptimal index usage.

5. Database Design and Normalization: A well-designed and normalized database structure can significantly impact query performance by minimizing data redundancy and improving data integrity.

6. Limit Data Retrieval: For queries returning large result sets, use LIMIT and OFFSET to retrieve data in smaller, manageable chunks, especially in paginated applications.

7. Avoid Anti-Patterns: Minimize the use of self-joins, cross-joins (unless absolutely necessary), and DML statements that update or insert single rows in a loop. Consider bulk operations where possible.

8. Prefer UNION ALL over UNION: If you know your combined result sets will not have duplicates, use UNION ALL. UNION adds overhead by performing an implicit DISTINCT on the results.

9. Use EXISTS instead of IN for subqueries:

Bad (pseudocode):

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);
```

Good (pseudocode):

```
SELECT name FROM customers
WHERE EXISTS (
    SELECT 1 FROM orders WHERE orders.customer_id =
    customers.customer_id
);
```

Why This Matters:

EXISTS stops searching once a match is found

Often better optimized by query planners

Reduces memory use for large subqueries

10. Replace nested subqueries with CTEs

11. Avoid WHERE YEAR(joining_date) = 2022 and use WHERE joining_date >= '2022-01-01' AND joining_date < '2023-01-01';

12. Partition and shard large tables:

13. Hardware Resources:

CPU and Memory: Ensure adequate CPU and memory resources are available for database server.

Disk I/O: Use SSDs for faster data access and reduce the latency.

14. Monitoring and Maintenance:

a. Regular Monitoring:

Performance Counters: Monitor key performance counters like CPU usage, memory usages, and I/O statistics.

b. Maintenance Tasks:

Update Statistics: Regularly update statistics to help the query optimizer make better decisions.

Rebuild Indexes: Schedule regular index rebuilds or reorganizations.

Database Cleanup: Remove unused data and reclaim space.

Command:

set statistics time on

select * from sys.dm_exec_connections

select * from sys.dm_exec_sessions

select * from sys.dm_tran_locks